WEEK 3

Storage for Scale: Distributed File Systems and NoSQL

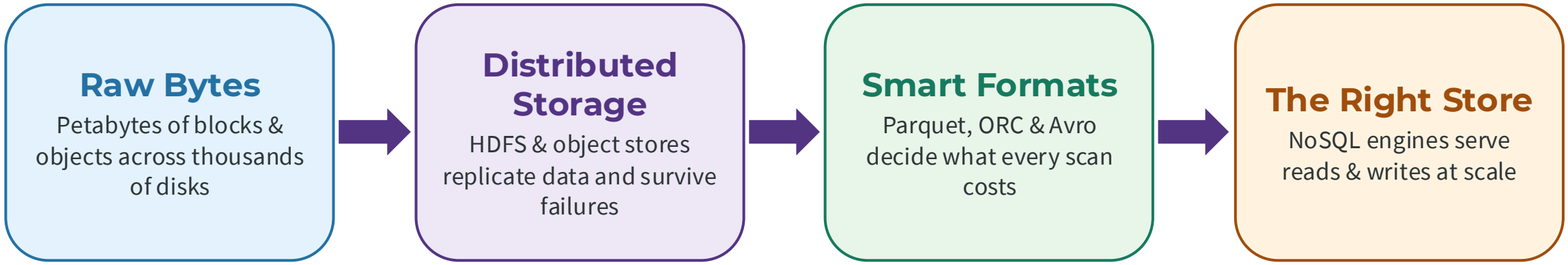
Files, Objects, Formats & NoSQL Stores Built for Petabytes

Distributed Storage
HDFS & S3-style object stores

Formats that Fly
Row vs Columnar
Avro, CSV, JSON, Parquet, ORC

NoSQL Stores
Key-value, document, wide-column & graph

From Bytes on Disk to Queryable Data



Storage is the foundation of the platform

Every engine we meet — Spark, Kafka, the lakehouse — stands on the storage decisions you learn this week.

Contents

Everything for week 3 — where data at scale lives, how it is laid out, and who serves it

01 Distributed Storage

Where 2 PB actually live
HDFS: blocks, replicas & the NameNode
Object storage, S3-style
HDFS vs object store — trade-offs

02 File Formats

Row vs columnar layout
Inside Parquet: pushdown & stats
ORC & Avro — when each wins
Choosing a format

03 The NoSQL Landscape

Key-value stores & **Dynamo**
Wide-column stores & **Bigtable**
Document & **graph** stores
Secondary indexes at scale
Choosing a store & the primary papers

04 Lab — Week 3

Format benchmarking on a real dataset
AI-assisted literature triage
Verify claims against the papers
Next week: MapReduce & Spark

From Principles to Disks

Week 2 gave you the theory of distribution — this week that theory hits real storage

What Week 2 Established

- Partitioning spreads data & load across machines
- Replication buys durability and availability
- Consistency models define what readers may see
- CAP & PACELC force explicit trade-offs

What Week 3 Builds on It

- File systems & object stores that apply those rules
- Formats that decide what every scan costs
- NoSQL engines that pick a side of CAP when partitions happen
- Indexes that trade write cost for read speed

This Week's Pillars

HDFS | **Object Storage** | **Formats** | **NoSQL** | **Indexes**

Scenario — Two Petabytes to Keep

Before anything is computed, the data has to live somewhere — safely, cheaply, and fast to scan

The Question

"Where do we put 2 PB so it survives failures, scans fast — and does not bankrupt the project?"

- Three years of clickstream: ≈2 PB in ~10^9 files — ≈2 MB each on average
- Commodity disks fail at **~1–2% per year**: with hundreds of disks, failure is routine
- Analysts read a few columns; the fraud team reads single records
- And every byte must stay durable, auditable and cheap to keep

The Napkin Math

```
# Store: keeping 2 PB on 16 TB disks:
#   ≈125 raw → 375 with 3× replicas, no headroom
#   1.5% AFR (assumed) → ≈5–6 dead disks/yr
# Scan: one 200 MB/s stream over 2 PB: ≈116 days
#   125 streams in parallel: ≈22 h (ideal)
the catch — someone must:
place replicas across racks,
  detect failures & re-replicate,
  track metadata for 10⁹ files,
  and lay bytes out for the queries
→ that is what storage systems do
```

The lesson

At this scale, failure is routine, and a full scan only becomes viable through parallelism and reading fewer bytes. Storage must replicate, detect and repair continuously — and the layout of every file decides what a query costs.

Notes: Bytes: KB, MB, GB, TB, PB. AFR: Annual Failure Rate. Headroom is the operability cost (metadata, query execution, free space, …) on top of base data, and it is what makes repair after failure possible at all. Rule of thumb in production: plan for 20–30% free, so the honest number is closer to ≈450–500 disks, not 375. Rack (cabinet) – holds 20 to 40 disks, share one top-of-rack (ToR) network switch and power unit. Rack failures are correlated. Replication must be on disks on distinct racks.

HDFS — Blocks, Replicas & the NameNode

Ghemawat, Gobioff & Leung, “The Google File System” (SOSP 2003), industrialised by Hadoop — write once, read many

1

Split

Files are cut into large blocks — 128 MB by default. Big blocks favour sequential throughput over seeks, and keep metadata small.

2

Replicate

Each block is copied to 3 DataNodes (default), rack-aware: two racks, three machines — with replicas healthy, a whole rack can fail safely.

3

Track

The NameNode holds the namespace and block map in RAM — the source of truth and a scaling limit. HA provides failover; Federation scales metadata across independent namespaces.

4

Serve & repair

Clients read blocks directly from DataNodes; heartbeats expose failures and background re-replication heals them.

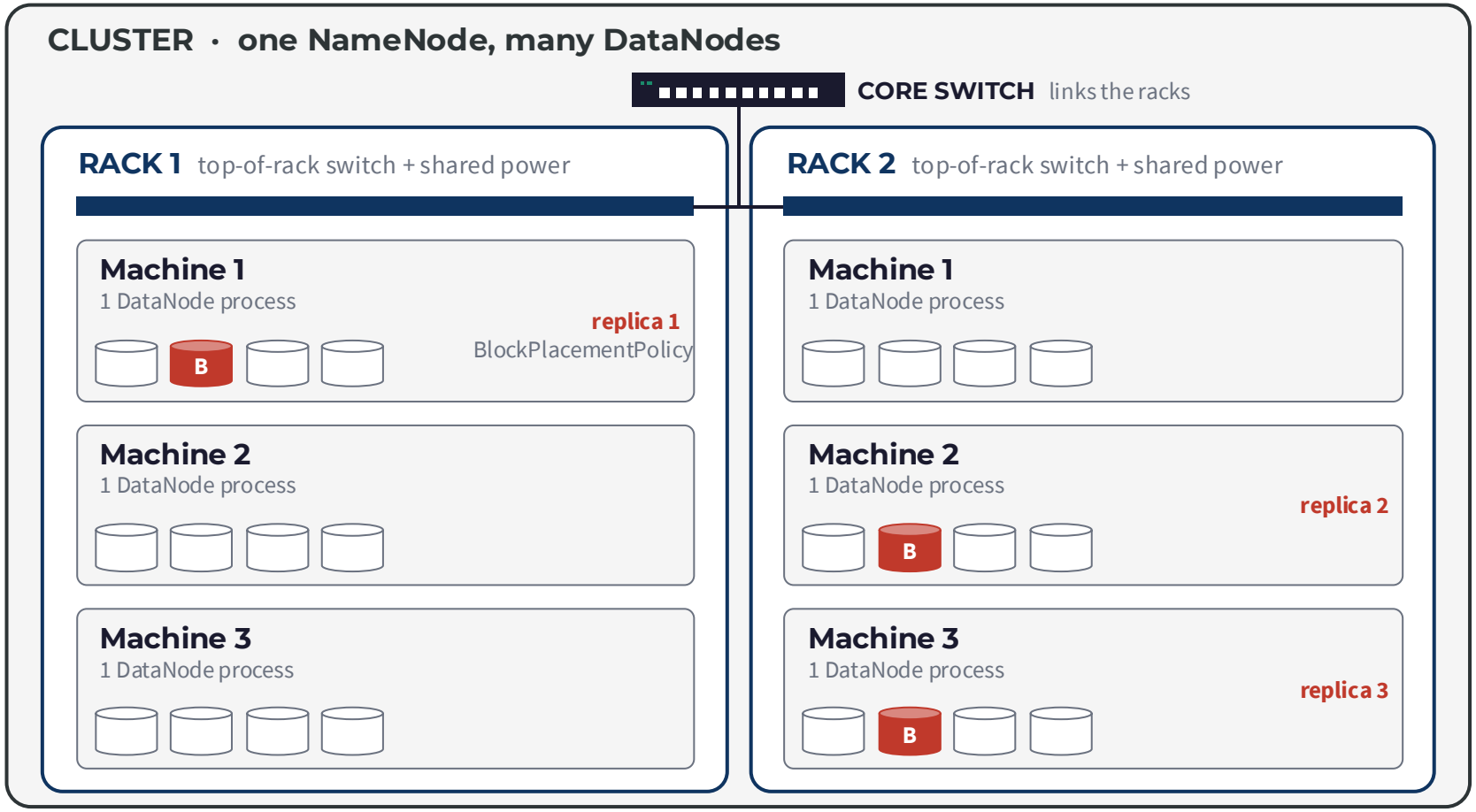
Why this matters

- HDFS optimises write-once, read-many: sequential scans at disk speed; no arbitrary in-place updates — exactly the access pattern of analytics.
- The same three ideas — blocks, replicas, a metadata service — reappear in every storage system we meet this week.

Notes. HDFS = Hadoop Distributed File System, an open-source GFS (Google File System) design: Shvachko et al., "The Hadoop Distributed File System", MSST 2010.
HA = High Availability (Hadoop 2+): Multiple (2 to 5) NameNodes, one Active and the rest Standby, sharing the edit log through a JournalNode quorum; ZooKeeper triggers automatic failover. HA removes the single point of failure but not the RAM limit; Federation does that, by running several NameNodes side by side, each owning its own namespace, while all of them share the same set of DataNodes.

HDFS topology — cluster → racks → machines → disks

Replicas are placed by machine and rack, never by disk — one DataNode process per machine



Cluster
All machines the NameNode manages; one namespace.

Rack
A cabinet of servers sharing one switch and power: a single failure domain.

Machine
One server = one DataNode daemon; never holds two replicas of a block.
Server + Disk controller + disks.

Disk
Several per machine; the DataNode spreads blocks over them. Placement (by the NameNode) chooses machines, not disks.

Block B, 3 replicas
Rack 1: writer's machine. Rack 2: two different machines. Lose a whole rack, one copy survives.

Source: Shvachko et al., "The Hadoop Distributed File System", MSST 2010, §3.2 — default BlockPlacementPolicy
HDFS default policy – 3 replicas – 1 in writer's machine, 2 on two distinct machines on another rack.

Object Storage — The S3 Model

1 — Objects & keys
whole-object writes in a flat namespace

2 — Durable by design
redundant copies across zones (S3 Standard)

3 — Storage as a service
an HTTP API: GET · PUT · LIST

No file system — just keys and an API

```
// a “path” is only a key prefix
s3://lake/events/date=2026-07-16/part-0007.parquet
GET fetches the object — whole, or a byte range
  PUT replaces the whole object — no partial edits
  → no append/rename in general purpose buckets, no directories
// exception: S3 Express One Zone adds append & RenameObject
// Amazon S3 Standard, general purpose buckets:
durability 99.999999999% designed, per year — 11 nines
reads: strong read-after-write (since Dec 2020)
  LIST is an API call — slow, and billed
  → design your layout around prefixes
```

Why it became the default

- ✓ Capacity is elastic — pay per GB-month
- ✓ Storage scales independently of compute
- ✓ Major analytics engines support S3 access
- ✓ Multi-zone redundancy in standard classes
- ✓ The data lake — and lakehouse — sit on it

A bucket is cheap, durable and boring — exactly what you want underneath everything else.

Source: Bornholt et al., "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in **Amazon S3**", SOSP 2021; Amazon S3 docs.

S3 — Inside an Object Store

A request never touches a disk directly: front-end → index → erasure-coded shards spread over three Availability Zones

Client
Spark job, laptop, app

`PUT lake/events/part-7.parquet`

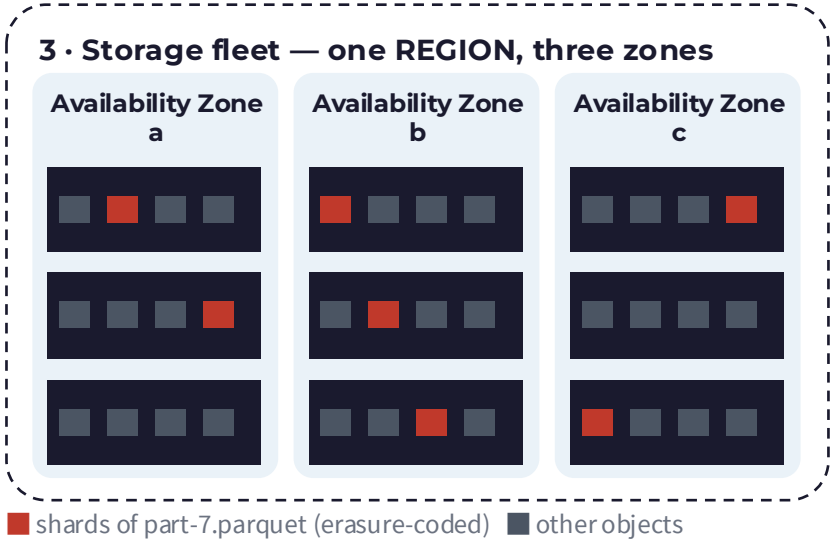
HTTPS, signed with IAM credentials

1 · Front-end fleet
REST endpoint (GET · PUT · LIST)

Authenticates & authorises the request, verifies the checksum, routes by bucket and key

2 · Index (metadata)
key → version, size, shard locations

Partitioned by key prefix — this is what LIST scans, and why there are no real directories



1 Request
Client sends HTTPS PUT with bucket, key and body; the SDK adds a checksum.

2 Front-end
Verifies identity and permissions; rejects or forwards. No disk touched yet.

3 Encode
Object is split into shards with parity (erasure coding), not copied 3× like HDFS.

4 Store
Shards are written to storage nodes in ≥3 AZs; acknowledged only after enough land.

5 Commit
Index records key → shards, then 200 OK. Every GET/LIST now sees it (strong consistency).

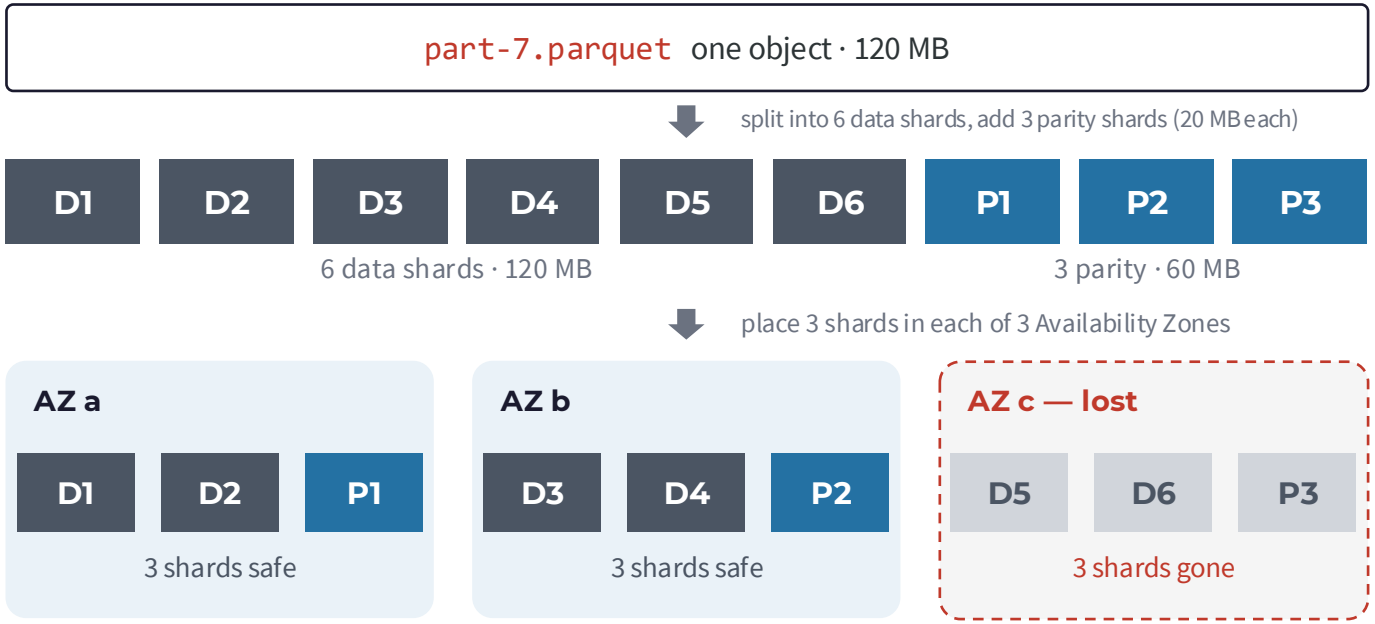
Contrast with HDFS: no NameNode and no DataNodes you can see — the index and the storage fleet are Amazon's, spread over zones; you only ever hold a key.

Source: Bornholt et al., SOSP 2021 (ShardStore); Amazon S3 documentation. Diagram is a simplified model, not Amazon's internal topology.

Notes: IAM - Identity and Access Management. AZ – Availability Zone: one group of data centres inside an AWS Region (e.g. eu-west-1), each with its own power, cooling and network, kilometres apart but linked at millisecond latency, so a fire or grid failure in one zone does not touch the others. Bucket & key: lake/events/part-7.parquet: bucket (a globally unique container, *lake*) and the key (an arbitrary string inside it, *events/part-7.parquet*)

S3 — Erasure Coding

6 data + 3 parity shards, any 6 rebuild the object: 1.5× storage instead of 3×, and a whole Availability Zone can vanish



	3× replication (HDFS)	6 + 3 coding (S3)
Storage overhead	3× (360 MB)	**1.5× (180 MB)**
Failures tolerated	2 copies	**3 shards (1 AZ)**
Cost of a read	**1 node, local**	6 nodes + decode
Cost of a repair	**copy 1 replica**	read 6, rebuild 1

6 of 9 shards remain, and any 6 rebuild the object. Every GET still succeeds; when AZ c returns or repair runs, the 3 missing shards are recomputed from the other 6.

P1, P2, P3 are three different combinations of D1–D6, not copies: each parity shard is a fixed, independent linear combination of D1–D6, in a way such that from any 6 of the 9 shards the system can recover the missing 3 ones.

Why HDFS still copies: MapReduce reads a block whole from a local disk, so one full replica per node is worth the bytes. S3 objects are read over the network anyway, so fetching 6 shards costs little extra. HDFS 3.x adds erasure coding for cold data for the same reason.

Contrast with HDFS: the same failures survived with half the bytes on disk; the price is that every read and repair touches 6 nodes, not one.

Source: Plank, “Erasure Codes for Storage Systems”, ;login: (2013); AWS re:Invent STG331 (2019). 6 + 3 is illustrative: Amazon does not publish S3's coding parameters.

HDFS vs Object Storage

Aspect	HDFS	Object storage (S3-style)	Edge (our 2 PB scenario)
Compute locality	`Data-local reads — code moves to disks`	Always over the network	HDFS
Elasticity	`Add nodes & rebalance yourself`	Elastic within service limits; managed for you	Object
Durability	`3× replication inside one cluster`	Provider-managed redundancy; multi-zone in S3 Standard	Object
Cost model	`Buy & run the cluster 24/7 — idle disks still cost`	GB-month + requests, transfer & retrieval by class	Object
Latency	`Local reads can avoid remote request overhead`	Tens of ms per request (indicative, S3 Standard); LIST hurts	HDFS
Ecosystem today	`On-prem Hadoop estates, legacy`	Default for cloud & the lakehouse	Object

`The modern default is object storage + open formats — HDFS thinking lives on inside it.`

On-prem estates and workloads that depend on local reads still run HDFS; the lakehouse we build in week 9 assumes a bucket.

Row vs Columnar

One question decides the layout: do your queries read **records** — or **columns**?

Row — one record at a time

```
# OLTP: fetch one order
SELECT * FROM orders
  WHERE id=4711
  → indexed lookup, whole record
  writes fast; indexes add cost
  reads drag 48 columns along
```

Columnar — a column at a time

```
# OLAP: revenue per day
SELECT day, SUM(total)
  FROM orders GROUP BY day
  reads 2 of 48 cols ≈ 4% I/O
  (if columns equal-sized)
  similar values adjacent →
  compress well, scan fast
```

The practical difference

```
                rows vs columnar
reads    = records vs aggregates
writes   = cheap   vs batched
compress = modest  vs excellent
best for = OLTP    vs analytics
```

Why columnar wins for analytics

- ✓ **Skip unread columns**
- ✓ **Strong compression**
- ✓ **Vectorised scans**

*Analytics touches a few columns of billions of rows — **columnar** layout turns that into sequential scans of exactly those bytes; **row** layout keeps single-record reads & writes cheap. Hence Parquet at rest, rows in OLTP.*

Source: Abadi, Madden & Hachem, "Column-Stores vs. Row-Stores: How Different Are They Really?", SIGMOD 2008; Kleppmann, DDIA ch. 3.

Notes: OLTP – Online Transaction Processing: many small, concurrent reads and writes that each touch one or a few whole records (place an order, update a balance). OLAP – Online Analytical Processing: few, large read-only queries that scan millions of records but only a handful of columns (revenue per day). What matters here is the access pattern, not the data: OLTP wants the whole record in one place (row layout); OLAP wants one column contiguous on disk (columnar layout).

Inside Parquet

```
# 2 TB of events — how much must a query read?
from pyspark.sql.functions import col
df = (spark.read.parquet("s3a://lake/events/")
  .filter(col("country") == "PT")  # pushdown
  .select("day", "total"))  # 2 of 48 cols
# footer: schema + min/max stats per chunk
# dictionary + RLE encode repeated values
# snappy / zstd compress pages in each chunk
# projection skips unused columns;
# stats may skip row groups; read cols
# output + predicates need — lab 3.1
```

The Vocabulary

- **Row group** → horizontal slice, ~128 MB–1 GB
- **Column chunk** → one column inside a row group
- **Page** → smallest unit of encoding & compression
- **Statistics** → min/max per chunk — enables skipping
- **Predicate pushdown** → stats may skip groups; rest filtered later
- **Projection** → read output + predicate columns

Parquet is the default format at rest for analytics — ORC pulls the same levers; Avro (next) is row-oriented and suited to record serialization; it lacks columnar projection savings.

Source: Apache Parquet format specification (parquet.apache.org); Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets", VLDB 2010 (nested columnar encoding).

Parquet Anatomy

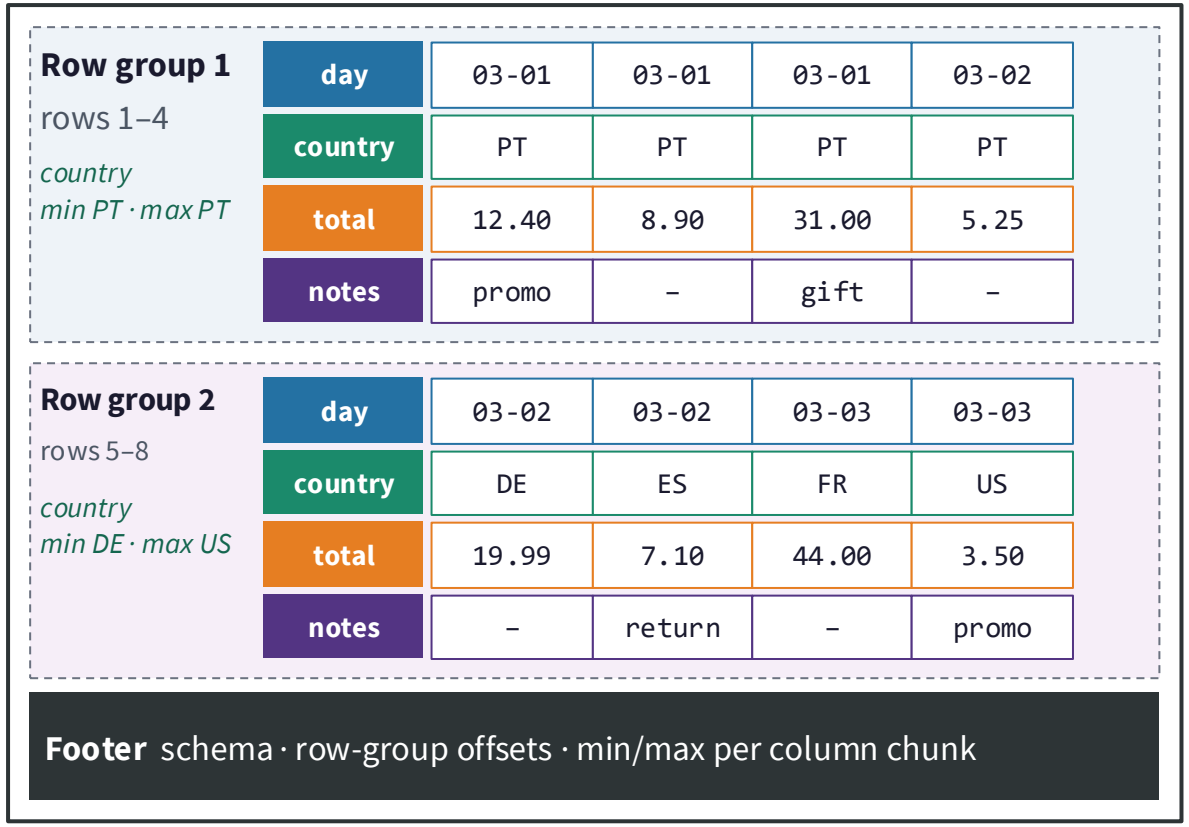
Logical table — 8 rows × 4 columns

day	country	total	notes
03-01	PT	12.40	promo
03-01	PT	8.90	–
03-01	PT	31.00	gift
03-02	PT	5.25	–
03-02	DE	19.99	–
03-02	ES	7.10	return
03-03	FR	44.00	–
03-03	US	3.50	promo

rows 1–4 → row group 1 **rows 5–8** → row group 2

Column chunk = one column of one row group · **page** = unit of encoding · reader opens the **footer** first, then seeks to the chunks it needs

The same 8 rows as one Parquet file



Columnar within a row group: rows never straddle groups, so one row group = one Spark task and one unit of skipping.

Source: Apache Parquet format specification, "File format" (parquet.apache.org/docs/file-format). Values illustrative; real row groups hold 10^5–10^7 rows.

Parquet — What a Query Actually Reads

`SELECT day, total FROM events WHERE country = 'PT'` · 2 TB, 48 columns, 16 000 row groups of 128 MB

2 TB

Stored

Whole table on S3. Nothing read yet except the footer: schema, row-group offsets, min/max per chunk. One seek.

÷10 →

200 GB

Prune row groups

Footer stats: 90 % of row groups have no 'PT' in country → skipped without reading. 1 600 groups left.

÷16 →

12.5 GB

Prune columns

Projection + predicate: read 3 of 48 column chunks (day, total, country). The other 45 stay on disk.

÷3 →

≈ 4 GB

Decode survivors

Dictionary, RLE, bit-packing + Snappy ≈ 3× smaller. Only now are bytes decompressed and scanned.

0.2 % of the table is read — and the pruning happens *before* any byte is decompressed.

Statistics prune only clustered columns

Write events sorted or partitioned by country and min/max ranges are narrow. Random order → every row group contains 'PT' → step 2 saves nothing, 2 TB becomes 125 GB, not 4.

Files are immutable — no in-place update

Footer offsets and stats are written once at close. Changing a row means rewriting the file; Delta Lake and Iceberg exist to manage that on top of Parquet.

Same three levers as slide 12: skip row groups, skip columns, decode compactly — one row group per Spark task is also where the parallelism comes from.

Illustrative numbers: equal-width columns, 90 % of row groups pruned, ≈3× encoding + Snappy. Source: Apache Parquet format specification (parquet.apache.org).

The Format Family at a Glance

The formats

Parquet
columnar — the analytics default

ORC
columnar — Hive heritage; used by Hive ACID tables

Avro
row-based — schema evolution champion

CSV / JSON
text rows — easy, expensive at scale

Arrow
columnar in memory — zero-copy where supported

Where each one fits

Analytics at rest
Parquet & ORC: statistics + compression make scans cheap

Pipelines & Kafka
Avro rows with a schema registry evolve safely over time

Ingest & interchange
CSV/JSON land raw data — convert once, never scan twice

Between engines
Arrow hands columns between compatible engines, often without copies

`Rule of thumb:` *row formats move records, columnar formats serve scans — define and validate schemas explicitly in the pipeline.*

Source: Apache Parquet, ORC, Avro and Arrow project documentation; Kleppmann, DDIA ch. 3–4 (storage layouts, encoding & schema evolution).

Notes: ORC – Optimized Row Columnar. ACID – Atomicity (all or nothing), Consistency (data stays valid), Isolation (concurrent transactions do not see each other's half-done work), Durability (a commit survives crashes). Hive ACID tables bring these guarantees to UPDATE/DELETE on ORC via delta files merged on read or compaction.

Choosing a Format

Format	Layout	Strength	Use it for
Parquet	`columnar`	pushdown, stats, ecosystem	Default
ORC	`columnar`	stripe indexes; Hive ACID tables	Warehouses
Avro	`row`	schema evolution, compact	Pipelines
CSV	`row · text`	universal, human-readable	Interchange
JSON	`row · text`	nested & flexible	Landing zone
Arrow	`columnar · in memory`	zero-copy interchange (where supported)	In memory

`No format wins everywhere — but Parquet is the default until a measurement says otherwise.`

…lab 3 makes this concrete: the same dataset in four formats, five encoding/compression configurations — size on disk, one-column scan, selective query.

Source: project documentation of each format; Abadi et al., SIGMOD 2008 for the row/columnar trade-off. Rankings are guidance — measure on your own data (Lab 3.1).

Why NoSQL?

The relational model did not fail — single-node deployments met workloads they were never sized for

What relational gives you

```
ACID transactions — invariants held under concurrency
rich SQL — joins & aggregates over anything
schemas that guarantee the shape of every row
  /* decades of tooling & know-how */
one node is the classic deployment — easy to reason about
  /* …until data outgrows the box; distributed SQL (Spanner, CockroachDB) is one answer */
→ choose by workload & guarantees, not SQL vs scale
```

Where it strains at scale

- Joins across partitions are slow — or unsupported
- Single-node write throughput hits a hard ceiling
- Rigid schemas fight fast-evolving, nested data
- Manual sharding of a single-node DB becomes an ops burden

What NoSQL engines do differently:

Partition | **Replicate** | **Denormalise** | **Specialise**

NoSQL — The Map

The common recipe — every NoSQL engine makes these four trades

Partitioned by key
gains **horizontal scale** · gives up **cross-partition joins**

Replicated, often leaderless
gains **stays up through node loss** · gives up **a single consistency model — CP or AP is chosen**

Schema-on-read
gains **fast evolution, nested data** · gives up **DB-enforced shape — the app validates**

Single-record atomicity
gains **high throughput** · gives up **multi-row ACID — some products added it later**

Four ways to package it — what you address, and where the deck covers each

Key-value
`key → opaque value`

get / put by key, nothing else. Fastest, simplest; no ranges, no queries on content.

Dynamo, Riak, Redis (Redis: in-memory, not Dynamo-style) · *slide 20*

Wide-column
`(row key, column, ts) → value`

Rows sorted by key; columns sparse and dynamic. Range scans and huge write rates.

Bigtable, Cassandra, HBase · *slide 22*

Document
`key → nested JSON aggregate`

The whole order, profile or catalogue entry travels together; indexes on any field.

MongoDB, CouchDB, Firestore · *slide 27*

Graph
`nodes + edges + properties`

Relationships are first-class; traversals replace joins. Traversal cost grows with hops, not table size.

Neo4j, Neptune · *slide 30*

‘NoSQL’ names a family, not a technology — choose by access pattern (slide 12’s lesson again), not by fashion.

Source: Kleppmann, Designing Data-Intensive Applications (2017), ch. 2, 5–6; Cattell, “Scalable SQL and NoSQL Data Stores”, SIGMOD Record 2010.

Key-Value Stores — the Dynamo Recipe

Amazon, 2007 — built for a shopping cart that must always accept writes

1. Hash
Consistent hashing places keys on a ring

2. Spread
Virtual nodes balance; N replicas clockwise

3. Quorum
R + W > N tunes consistency per call

4. Heal
Hinted handoff & anti-entropy repair

R and W — pay for freshness only where you need it

```
N = 3  // home replicas of the key
W=2, R=2  R+W=4 > 3 → every read overlaps the write
          …only if write and read use the same 3 nodes;
          if one is down, Dynamo writes to a hinted
          replica (sloppy quorum) → a read can miss it
W=1, R=1  R+W=2 < 3 → write lands on 1 node;
          2 of 3 reads return the old value
W=1, R=2  R+W=3 = 3 → no overlap guaranteed:
          the 2 read may be the 2 that missed it
lower W or R → faster, tolerates more dead nodes
// concurrent writes still diverge →
//   vector clocks detect; LWW/app resolves
```

Traits & descendants

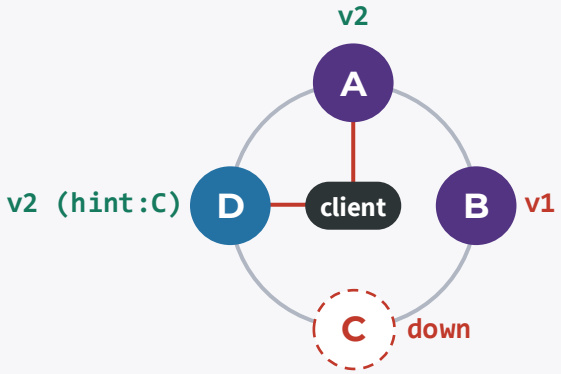
- **AP by design** — aims to stay writable while enough nodes are reachable; reconciles after the partition heals
- **Eventual consistency** — stale reads are a documented feature, not a bug
- **Hash lookups only** — no ranges, no joins — low latency, set by storage, replication & network
- **Descendants** — influenced DynamoDB, Cassandra, Riak; today's products differ from the 2007 paper
- **Not a descendant: Redis** — in-memory, one leader per shard, rich value types; the everyday key-value store, but not the Dynamo recipe

Notes: Vector clock – a version tag with one counter per replica, e.g. [A:2, B:1]. If every counter of one tag ≥ the other's, it is the newer write; otherwise the writes were concurrent and Dynamo returns both for the app to merge. LWW – Last Writer Wins: keep the latest timestamp, drop the rest. AP – Availability during Partition.

Sloppy Quorums

R + W > N is not enough: Key K lives on home nodes A, B, C (N = 3), D is next on the ring; W = 2, R = 2, and C goes down.

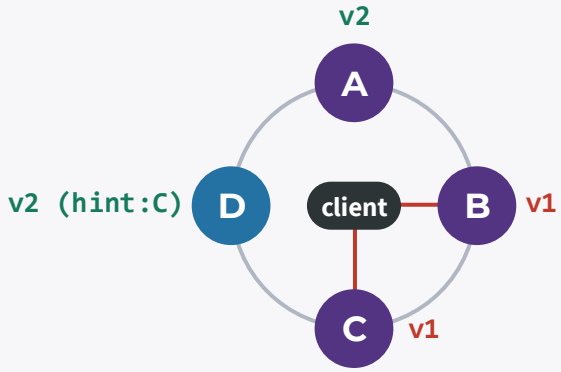
1 Write while C is down



client writes K = v2, W = 2. A acks; C is unreachable, so the coordinator writes to D with a hint “owner: C”. 2 acks → success.

sloppy quorum: 2 acks, not all from home nodes

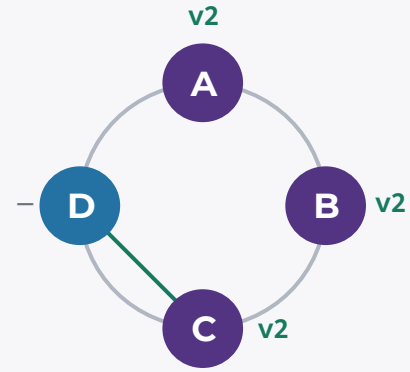
2 Read before handoff



C is back; D has not yet delivered. Client reads K, R = 2, from the home nodes and hits B and C: both return v1.

R + W = 4 > 3, yet the read missed v2

3 Repair



D hands v2 to C and drops its copy (hinted handoff). Merkle-tree comparison between A, B, C fixes what handoff missed (anti-entropy).

converges after repair, not at write time

Strict quorum

W acks from home nodes only; if too few are reachable the write fails.

Sloppy quorum

W acks from any reachable nodes; the write never fails — Dynamo’s choice.

Hinted replica

A non-home node holding a copy plus the owner’s identity (the hint).

Hinted handoff

The hinted replica delivers the copy to its owner when it returns, then drops it.

Dynamo chose “always writable” over “always fresh”; repair is the price. Cassandra’s QUORUM is strict; Riak has a sloppy_quorum flag.

Source: DeCandia et al., “Dynamo”, SOSP 2007, §4.6–4.7 (hinted handoff, anti-entropy); Kleppmann, DDIA ch. 5, “Sloppy quorums and hinted handoff”.

Wide-Column — Bigtable & LSM

Google, 2006 — a sorted, sparse, versioned map spread over thousands of servers; the parts first, the moving picture on the next slides

The data model — a cell; a "row" is a set of cells

```
Cell: (row key, family:qualifier, timestamp) → value
```

row key	user:name	user:country	cart:items
ord-4711	@t3 Ana	@t3 PT	@t3 [A7, B2]
ord-4712	@t1 Ben	@t1 DE · @t2 FR	—
ord-4713	—	@t5 ES	@t5 [C9]
ord-4714	@t2 Dan	@t2 US	@t2 [A7]

A map, not a table: the drawing is for us; on disk there are only cells sorted by (row, column, time). **Sparse:** a missing cell costs nothing. **Versioned:** several versions per cell (@t). **Sorted:** neighbouring keys sit together on disk. **Dynamic columns:** cart:items could be one column per item (cart:A7 → 2); qualifiers are per row. LSM = Log-Structured Merge-tree

Tablets — how a sorted table is cut up and served

A **tablet** = one contiguous key range, ~100–200 MB, served by exactly one tablet server. Adjacent keys sit in the same tablet, so a range scan usually hits one tablet.

all rows, sorted by row key →



Too big → split



Past the size limit the server cuts it at a middle key; both halves stay put until the master rebalances. Rows are sorted, so a split is one boundary.

Too busy → move



The master reassigns it to a quieter server. Its files live in GFS, visible to every server, so the move copies no data: only metadata changes.

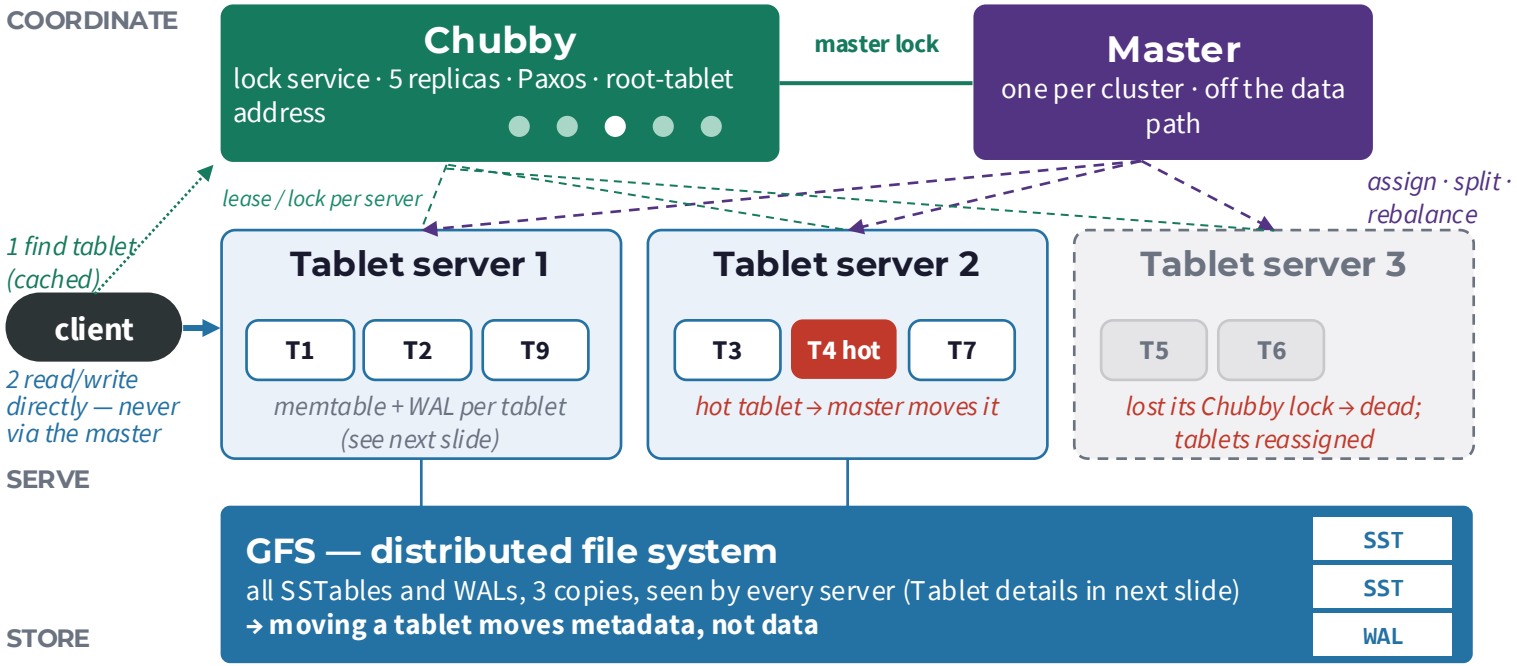
Row-key design follows: adjacent keys are cheap to scan together.
A monotonically increasing key (timestamps, sequence numbers) sends every insert to the last tablet and one server takes all the load. Salt, hash-prefix or reverse the key.

Two rules explain the design: one row (all its cells) is the unit of atomic writes (no multi-row transactions) · **one tablet is the unit of distribution** (one server, split when big, moved when hot)

Source: Chang et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006, §2, §5.3–5.4; O'Neil et al., "The LSM-Tree", Acta Informatica 1996.

A Bigtable Cluster

Who serves, who assigns, who stores: many tablet servers, one master, shared files, and a small lock service.



Chubby — the lock service

- Highly available lock + tiny-file service; five replicas, one leader, Paxos underneath
- Sessions with leases: a silent server loses its lock and is declared dead
- Holds the master lock, live-server list, root-tablet location, schema
- Apache twin: ZooKeeper (HBase, Kafka, Hadoop HA)

Master — the administrator

- Assigns tablets; reassigns on server death or a hot tablet
- Detects dead servers via their Chubby locks; registers splits
- Schema changes; garbage-collects unreferenced SSTables in GFS
- Off the data path: clients talk to servers directly; a master outage stalls only reassignment

How a client finds a tablet — three hops, all cached:

Chubby file	→	root tablet	→	METADATA tablet	→	**user tablet → its tablet server**
the one fixed address: where the root tablet lives		*first tablet of METADATA: where the other METADATA tablets are; never split*		*a Bigtable table, one row per "user" tablet → its key range and server*		*cached; a stale entry costs a retry, not an outage*

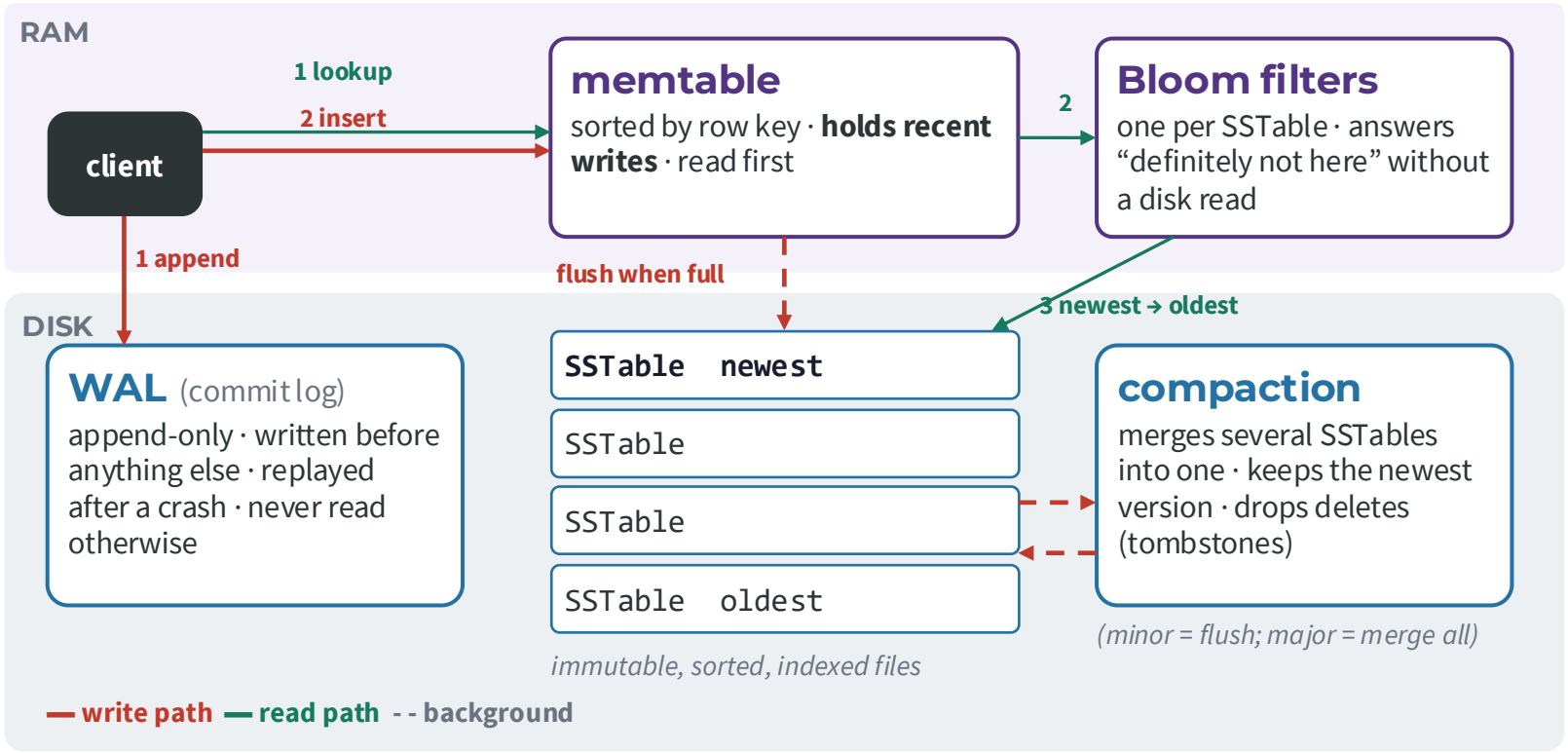
HBase, the open-source twin you can run: tablet = region · tablet server = RegionServer · master = HMaster · GFS = HDFS · Chubby = ZooKeeper

Paxos – consensus (Lamport): replicas agree on a value, then a sequence, despite failures, while a majority (3 of 5) answers; Chubby's five replicas thus hold one lock table. **Raft** solves the same problem (Raft lecture). Bigtable runs no consensus itself; it borrows Chubby's for what must be unanimous: who is master, who is alive.

Source: Chang et al., "Bigtable", OSDI 2006, §4 (building blocks: GFS, Chubby), §5.1–5.2 (tablet location & assignment); Burrows, "The Chubby lock service for loosely-coupled distributed systems", OSDI 2006.

Tablet Server — the LSM Engine

The five parts (right) and how they are wired (left): follow the red write path, then the green read path



The five parts

WAL disk	**Write-Ahead Log** (Bigtable: commit log). Append-only; replayed after a crash, never read otherwise.
Memtable RAM	Sorted buffer of recent writes, read first; flushed to an SSTable when full.
SSTable disk	**Sorted String Table**: immutable, sorted, indexed file; many per tablet, newest first.
Bloom filter RAM	One per SSTable; says "definitely not here" so most files are skipped without a disk read.
Compaction bg	Background **merge** of SSTables: drops old versions and tombstones, keeps the file count low.

LSM = Log-Structured Merge-tree
Every write is an append; every read is a merge. That inversion of the B-tree trade is the whole LSM idea.

Source: Chang et al., "Bigtable", OSDI 2006, §5.3–5.4 (tablet serving, compactions); O'Neil et al., "The Log-Structured Merge-Tree", Acta Informatica 1996.

LSM — the Paths, Step by Step

the write and read paths — step by step

```
## model — (row key, column, timestamp) → value
## rows sorted by row key — range scans are cheap
## a write:
  1. append to the WAL  // durable
  2. insert into the memtable  // sorted, in RAM
## when the memtable fills:
  - flush it as an SSTable — immutable
  - compaction merges files in the background
## a read:
  1. pick candidate SSTables — indexes, bloom filters
  2. merge their data with the memtable
  3. apply version filter — latest or N versions
# random writes become sequential appends
```

What to notice

✓ **Writes are appends**
The LSM-tree turns random writes into sequential I/O — enormous ingest rates

✓ **Reads pay the bill**
Bloom filters (next slide) and compaction keep a point lookup from touching every file; range scans still merge several files

✓ **Compaction is the bill**
Background merges cost I/O and CPU; skip them and reads slow down. Tuning an LSM engine moves the cost between writes, reads and space; it never removes it

Bigtable (2006) is the CP-leaning blueprint behind HBase, Cassandra's data model, ScyllaDB and the LSM engines (RocksDB, LevelDB) inside many modern stores.

Source: Chang et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006, §4–5; O'Neil et al., "The Log-Structured Merge-Tree (LSM-Tree)", Acta Informatica 1996.

Bloom Filter — a Cheap, Certain “No”

A guest list that fits in your pocket: it can never miss a real guest, but it sometimes waves in a stranger

m = 16 bits, k = 3 hash functions. Insert: set the key’s k bits to 1. Query: any of its k bits 0 → definitely absent; all k bits 1 → maybe present.



No false negatives

If any bit is 0, the key was never inserted. A “no” is certain — that is why a read can skip the file.

Some false positives

Bits get shared as the array fills. Tune m bits per key and k hashes: ~10 bits/key, k = 7 → about 1 % false “maybe”.

Tiny and in RAM

A 1 GB SSTable with 10 M keys needs ~12 MB of filter. Built once when the file is written; never updated (SSTables are immutable).

No deletes, no listing

You cannot clear a bit (others share it) or ask “what is in here” — only membership.

In the LSM engine: a point read checks the memtable, then asks each SSTable’s filter; only files that say “maybe” are opened. Also in Cassandra, HBase, RocksDB, Parquet page indexes, browsers (unsafe-URL lists) and CDNs.

Source: Bloom, “Space/Time Trade-offs in Hash Coding with Allowable Errors”, CACM 1970; Chang et al., OSDI 2006, §6 (Bloom filters on SSTables). Positions illustrative.

Document Stores — Aggregates

The model

- ✓ **JSON-like documents**
  Nested structure travels with the record — no assembly joins on read
- ✓ **Flexible schema**
  Fields vary per document; still validate and plan compatibility
- ✓ **Query by content**
  Secondary indexes on any field; rich predicates & aggregations

Aim: one aggregate = one read = one write

orders — one aggregate per document

```
{  // JSON-like sketch — the whole order together
  "_id": "ord-4711",
  "user": {"id": 17, "country": "PT"},
  "items": [{"sku":"A7","qty":2}, …],
  "total": 129.90
}  // fields differ per doc — fine

db.orders.find({"user.country":"PT",
  "total": {"$gt": 100}})

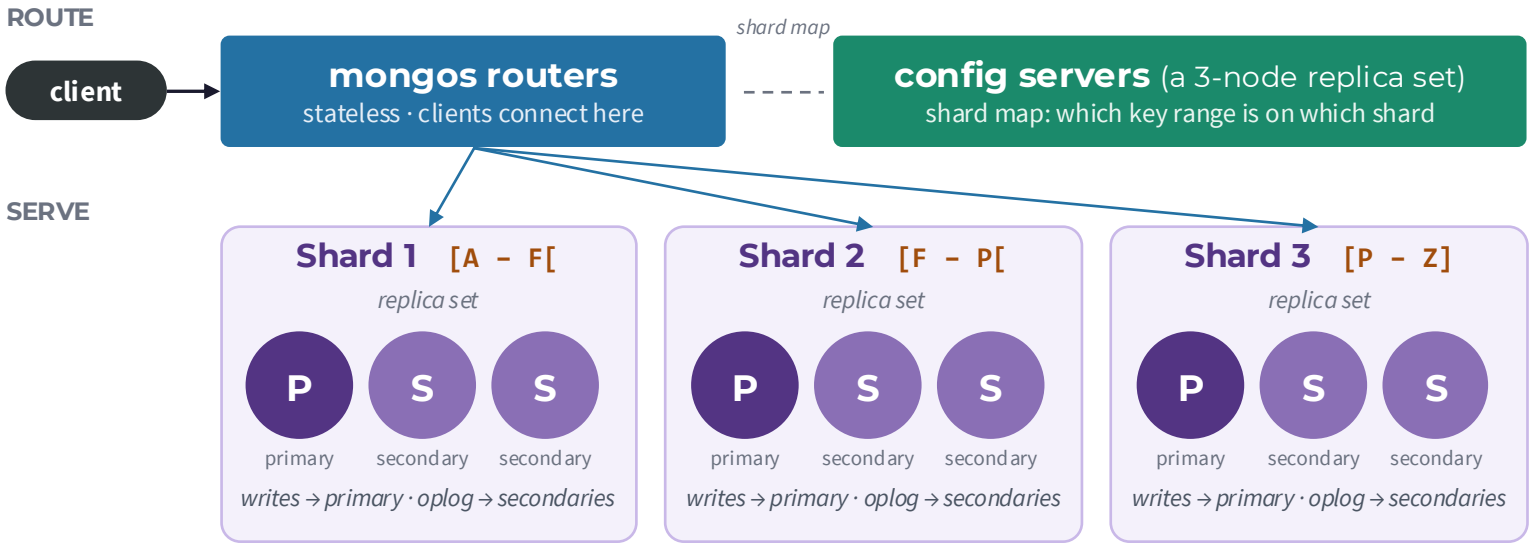
db.orders.createIndex({"user.country":1,"total":1})
  …find({…}).explain("executionStats") // plan and stats
  beware: unbounded arrays & hot docs
  need joins? denormalise — or rethink
```

MongoDB | **CouchDB** | **Firestore** | **RavenDB** | **Postgres (JSONB)**

- *Warning signs: **cross-document joins** and **fan-out writes** → redraw the aggregate, or use a relational or graph store.*
- *Embed only **bounded lists** (line items, addresses); an **unbounded** one-to-many gets its own collection with a back-reference.*
- ***Hot docs** pin load to one shard → bucket by time (create a new document per time window (per minute, hour or day)), or split into counters summed on read.*

MongoDB — the Cluster

Same three jobs as Bigtable — route, serve, coordinate — but each shard is a small leader-based replica set, and the storage engine is a B-tree.



If primary dies, a majority elects the secondary with the newest oplog (Raft-like); writes pause for seconds, reads go on.

Oplog = operations log; with write concern w (MongoDB's write quorum) a write is acknowledged once w replicas have applied it. Config servers replicate the same way.

INSIDE ONE NODE

WiredTiger B-Tree storage engine — *B-tree, not LSM: in-place updates, reads one path down*

BSON documents	secondary indexes	journal (WAL)	cache
in a B-tree by _id	B-trees on any field	append, then apply	RAM, checkpoints to disk

Write path: journal → B-tree + oplog in one WiredTiger transaction → secondaries replay the oplog

BSON = typed binary JSON: length-prefixed fields, native int / double / date / ObjectId types; ≤ 16 MB per document

Where a document lives

Whole document in one place (≤ 16 MB). The shard key picks the shard; _id picks the spot in its B-tree. Hence one aggregate = one read = one write.

Same roles as Bigtable

mongos ≈ client library + METADATA lookup
config servers ≈ Chubby + METADATA
shard ≈ tablet server
shard key range ≈ tablet range
New: each shard is an elected-primary replica set.

Where it differs

Leader per shard, not leaderless (Dynamo) and not one server per range (Bigtable). Storage is a B-tree: writes cost random I/O, reads are one lookup; no compaction bill.

One line each: Dynamo — leaderless, any replica writes, reconcile later · Bigtable — one server per range, LSM · MongoDB — one elected primary per shard, B-tree.

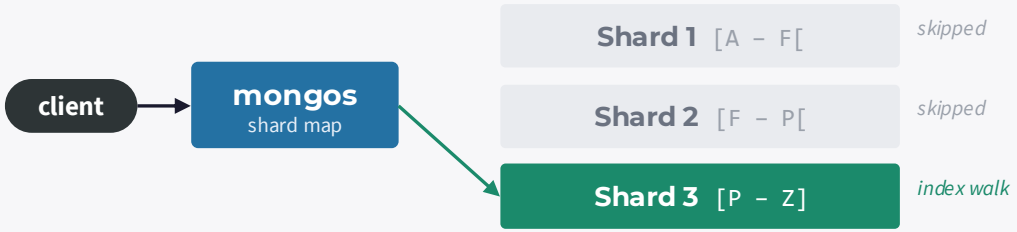
MongoDB shown; CouchDB (append-only B-tree, MVCC, master-master replication) and Firestore (on Spanner) differ in storage and replication. Source: MongoDB Manual — Sharding, Replication, WiredTiger; Kleppmann, DDIA ch. 5–6.

MongoDB — One Query's Path

Shard key = user.country. The same find() takes two very different paths depending on whether the query names the shard key.

A Targeted — the query has the shard key

```
db.orders.find({"user.country": "PT", "total": {"$gt": 100}})
```

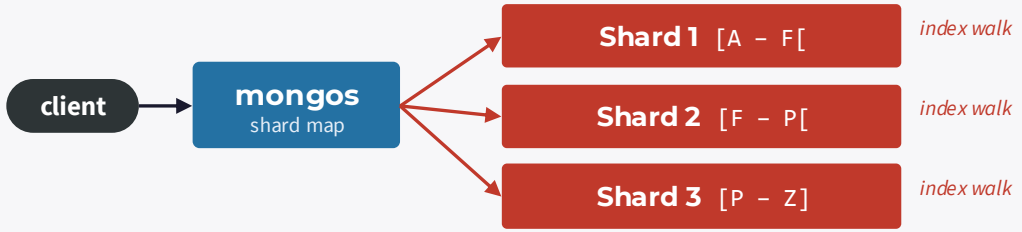


mongos reads the shard map: PT ∈ [P – Z] → Shard 3 only. On Shard 3 the primary uses the compound index (user.country, total), reads the matching documents once, returns them. One shard, one index walk.

cost grows with the result, not the cluster

B Scatter-gather — the query lacks the shard key

```
db.orders.find({"total": {"$gt": 100}})
```



mongos cannot tell where the documents are, so it sends the query to every shard, waits for all, merges (and sorts) the results. Each shard still uses its own index, but the slowest shard sets the latency.

cost grows with the number of shards

A write, step by step

mongos routes by shard key → the primary appends to its journal, applies to the B-tree, appends to the oplog → secondaries replay it. writeConcern: majority waits for a quorum before acking.

Reads: fresh or spread

readPreference: primary (default) → always the latest write. secondary → offloads the primary, may lag by the replication delay. Chosen per query, like Dynamo's R, but with one leader, not a quorum.

Transactions & the shard key

Multi-document ACID since 4.0 (cross-shard 4.2), at a latency cost. The shard key is chosen once per collection: high cardinality, even spread, present in the hot queries; monotonic keys make one shard hot.

The rule: put the shard key in every hot query. With it, a query touches one shard and one index; without it, every shard answers and the slowest one sets the latency.

Source: MongoDB Manual — Sharding (targeted vs broadcast operations), Replication (write concern, read preference), Transactions; Kleppmann, DDIA ch. 6 (partitioning and secondary indexes).

Graph Stores — Relationships

THE MODEL
Nodes + edges
relationships are stored, not recomputed by joins

THE QUERY
Traversals
you ask about paths, not about tables

- **Property graph** nodes and edges carry key-value properties — typed, directed, indexed
- **Index-free adjacency** each node points at its neighbours — following a known edge is a pointer hop, not an index probe (in engines that use it)
- **Built for** fraud rings, recommendations, knowledge graphs, data lineage

- **Query example:** a candidate pattern (in **Cypher**):

```
MATCH (a:Card)-[:USED_ON]->(d:Device)
      <-[:USED_ON]-(b:Card)
WHERE a.owner <> b.owner
WITH d, collect(DISTINCT a) AS cards
WHERE size(cards) >= 5 RETURN d, cards
```

Reads: one device, five or more cards of different owners — honest users rarely share a device; a shortlist for scoring, not a verdict.

- **Cost follows the walk** grows with vertices and edges visited — degree and filters matter; SQL can do it with joins or recursive queries — compare real plans
- **The caveat** partitioning a graph across machines is hard — edges cross partitions; scale-out remains the frontier

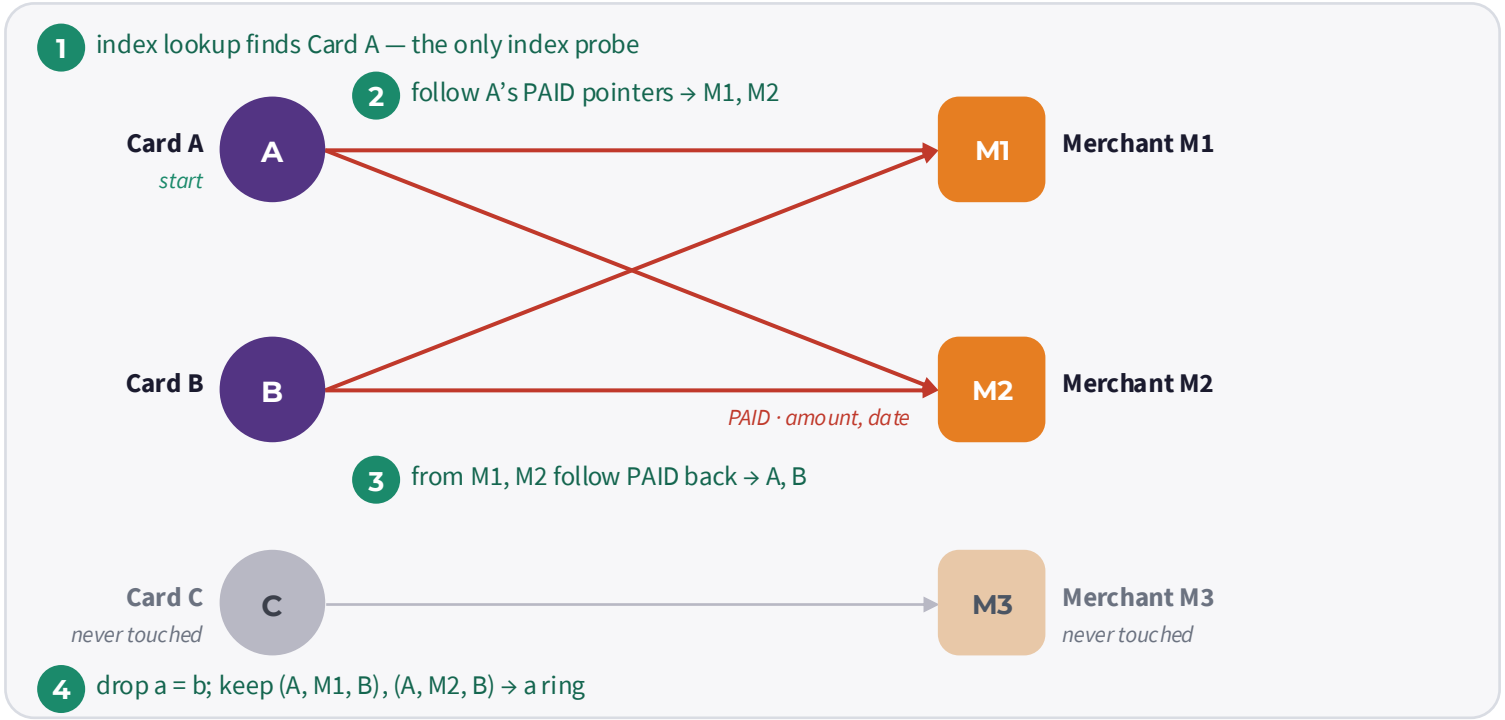
Rule of thumb: ***when relational queries become joins of joins of joins, evaluate a graph engine — and validate on representative traversals.***

Engines: Neo4j, Amazon Neptune, JanusGraph, or graph layers over stores you already met · **Language:** Cypher, Neo4j's, now ISO GQL.

Graph Stores — a Traversal

Two cards paying the same merchants, hop by hop — the shape of a collusive-merchant ring. One index lookup, then only pointers.

```
MATCH (a:Card)-[:PAID]->(m:Merchant)<-[:PAID]-(b:Card) WHERE a <> b RETURN a, m, b
```



Index-free adjacency

After hop 1 every step is a pointer from a node to its own edge list. No index is consulted again; a hop costs the same in a graph of 10^3 or 10^9 nodes.

Cost follows the walk

Work = edges visited: deg(A) × avg deg(merchant). Independent of how many cards exist in total. A merchant with 10^6 payments still explodes — degree, not table size, is the risk.

The same question in SQL

payments p1 JOIN payments p2 ON p1.merchant = p2.merchant WHERE p1.card <> p2.card. Equal at one hop with an index; the graph wins as depth grows; both suffer on supernodes.

Rule: pay once to find the start node, then only for the edges walked — the right bill for "who is connected to whom, through what".
Collusive merchants: merchants complicit in the fraud, where many suspicious cards converge

Source: Robinson, Webber & Eifrem, Graph Databases, 2nd ed., O'Reilly 2015, ch. 6 (index-free adjacency); Neo4j Cypher Manual. Two shared merchants are common; a candidate for review, not proof of fraud — count shared merchants and add time to make it selective.

Graph Stores — Storing Pointers

Neo4j as the example: fixed-size records linked by ids. “Neighbours of A” is read by following pointers, never by searching.



neighbours of A: node A → r1 (M1) → A’s next: r2 (M2) → ∅ — two pointer hops, zero index reads; M1’s chain r1 → r3 is walked the same way

ALSO ON DISK

property store
key–value chains hung off node and relationship records; read only when a property is asked for

label / property index
B-tree or text index to find the start node (:Card {id}) — the one index probe of a traversal

Why a hop is O(1): record id × record size = disk offset; no search, no hashing.

What it costs: every edge is doubly linked twice; an update touches several records; a supernode is one very long chain.

Scale-out: partitioning cuts edges, so Neo4j scales up and replicates; Neptune and JanusGraph shard over a key-value / wide-column store and pay an index probe per hop.

Rows vs pointers

A relational row stores a foreign key: a value you must look up. A relationship record stores an id: an offset you can jump to. Same information, different cost.

What the layout buys

Traversal cost ∝ edges walked, not graph size; adjacency lists come for free from the prev/next chains; properties stay out of the way until asked for.

What it gives up

Write amplification (several records per edge), no natural partition key, supernode chains, and a storage engine shared by no other system in this deck.

Four engines, four layouts: Dynamo hashes keys onto a ring · Bigtable sorts rows into tablets · MongoDB B-trees documents by _id · Neo4j links fixed-size records

Source: Robinson, Webber & Eifrem, Graph Databases, 2nd ed., ch. 6 (native graph storage: node, relationship and property stores); Neo4j Operations Manual (record formats). Record sizes are the standard-format values and illustrative.

Secondary Indexes at Scale

✓ Why this gets hard

- **Data is partitioned** — by primary key; other attributes scatter everywhere
- **A query on e-mail** — must find rows that were placed by user-id
- **Two designs exist** — index beside each partition, or index as its own table
- **Every index** — is another copy of keys & projections to keep in sync
- **Writes pay, reads gain** — you choose where it hurts

	No index query = scatter-gather	Global index partitioned by indexed value	Local index beside the data, per partition
READS	✗ scans every partition	✓ routes by key; ranges fan out	✓ needs partition key; else fan-out
WRITES	✓ cheapest possible	✗ cross-node; often async	✓ local; may be atomic
SYNC	✓ nothing to maintain	✗ often eventual (e.g. GSI)	✓ can be in step (e.g. LSI)

An index is a partial replica — keys plus projected fields, kept in sync on your behalf.
DynamoDB GSIs & LSIs, Cassandra's legacy 2i and Mongo indexes each sit somewhere on this line.

GSI = Global Secondary Index: its own partition key, any query shape, eventually consistent. LSI = Local Secondary Index: same partition key, alternative sort key, strongly consistent, fixed at table creation.

Choosing a Store — a Field Guide

LOOKUPS	AGGREGATES	WRITE-HEAVY	RELATIONSHIPS
Key-value Sessions, carts, caches — one key, one value, low-latency lookups	**Document** Read & write whole business objects with evolving shape	**Wide-column** Time series, IoT, feeds — LSM appends at enormous rates	**Graph** Relationship-centric queries; validate on real traversals
ANALYTICS	**INVARIANTS**	**REALITY**	**THE RULE**
Columnar lake Scans over billions of rows — Parquet plus a query engine	**Relational** Transactions & joins — single-node or distributed SQL	**Polyglot** Real platforms combine several stores behind one pipeline	**Access first** Model your queries before you model your data

Start from the access pattern, not the data — the store is a consequence of the queries and the guarantees you need. When in doubt: relational first; scale out when measurements demand it.

Reading List — Primary & Secondary

Three papers carry this week — triage with an assistant, verify by hand; the rest deepen one slide each. §-numbers point to what the deck uses.

1 · PRIMARY READING read before Lab 3

Dynamo

Problem: a shopping cart that must always accept writes

DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007

§4 techniques & Table 1 · slides 15–16, 21 · Lab 3.2

Bigtable

Problem: structured storage over GFS across thousands of nodes

Chang et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006

§4–5 tablets, SSTables, LSM path · slide 17 · Lab 3.2

GFS

Problem: huge append-mostly files on commodity hardware where failure is normal

Ghemawat, Gobioff & Leung, "The Google File System", SOSP 2003

§2 design, §4.2 replica placement · slides 6–7, 10

Optional framing: Brewer, "CAP Twelve Years Later", IEEE Computer 2012.

2 · SECONDARY READING one slide each, read as needed

HDFS *slides 6–7*
Shvachko et al., MSST 2010 — §3.2 block placement; NameNode limits

S3 internals *slides 8–9*
Bornholt et al., SOSP 2021 (ShardStore) — §2 only; rest is formal verification

LSM-tree *slide 17*
O'Neil et al., Acta Informatica 1996 — the structure behind Bigtable & Cassandra

Row vs columnar *slides 11–14*
Abadi, Madden & Hachem, SIGMOD 2008 — why columnar wins for scans

Nested columnar *slide 12*
Melnik et al., "Dremel", VLDB 2010 — the encoding Parquet borrows

Textbook *before Lab 4*
Kleppmann, DDIA ch. 3 (Storage & Retrieval) — covers LSM, B-trees, columnar in one pass

PART 2 · HANDS-ON

Lab

03